

PROJECT WORK CS6811
PROGRESS REPORT

**Federated Learning for Code Generation with
Hallucination-Aware Fine Tuning**

TEAM DETAILS

**Abhinavh P (2022503059),
Prabhakara Arjun R (2022503003),
Buyanes Srivardhan K (20225033037)**

MENTOR

**Dr. B. Thanasekhar,
Professor,
Computer Technology, MIT**

INTRODUCTION

Large Language Models (LLMs) have remarkable capabilities in code generation ,enabling automated solutions to a wide range of software engineering tasks and code generations. However LLMs frequently suffer from hallucinations in code generation, producing syntactically valid but semantically incorrect programs, non-existent APIs, or logically flawed implementations. Existing approaches to mitigating code hallucinations primarily rely on prompt engineering, repeated sampling, or full model fine-tuning. Full fine-tuning is computationally expensive and impractical in low-resource or decentralized settings, and repeated sampling does not address the underlying causes and mitigation of hallucination. In real-world development environments, code generation systems are often deployed across multiple decentralized clients where data cannot be centrally aggregated due to privacy, ownership.

To address these limitations we propose a hallucination-aware code generation framework that combines program analysis, targeted automatic program repair, and federated parameter-efficient fine-tuning using LoRA.

PROBLEM STATEMENT

Code generation using large language models (LLMs) is currently hindered by hallucinations. Existing mitigation strategies typically rely on supervised fine-tuning or reinforcement learning from human feedback (RLHF), where models are trained to align with human preferences. Parameter-efficient fine-tuning(PEFT) methods such as LoRA reduce computational overhead compared to full fine-tuning.

But in real world scenarios, high-quality code and execution feedback are distributed across multiple organizations making centralized data collection infeasible due to privacy concerns. Without a secure and decentralized learning mechanism, models are unable to effectively learn from diverse hallucination patterns observed across clients. This creates a gap for a unified framework that integrates explicit hallucination detection and mitigation framework in a federated learning setup with PEFT to improve the quality of code generation .

OBJECTIVE

- To fine-tune large language models using parameter-efficient such as LoRA in a federated learning setting.
- To evaluate the proposed framework on standard code generation benchmarks and analyze hallucination reduction.
- To design a hallucination-aware code generation framework that integrates static and dynamic program analysis.
- To apply automatic program repair for correcting hallucinated code using knowledge-guided patching.

ARCHITECTURE DIAGRAM

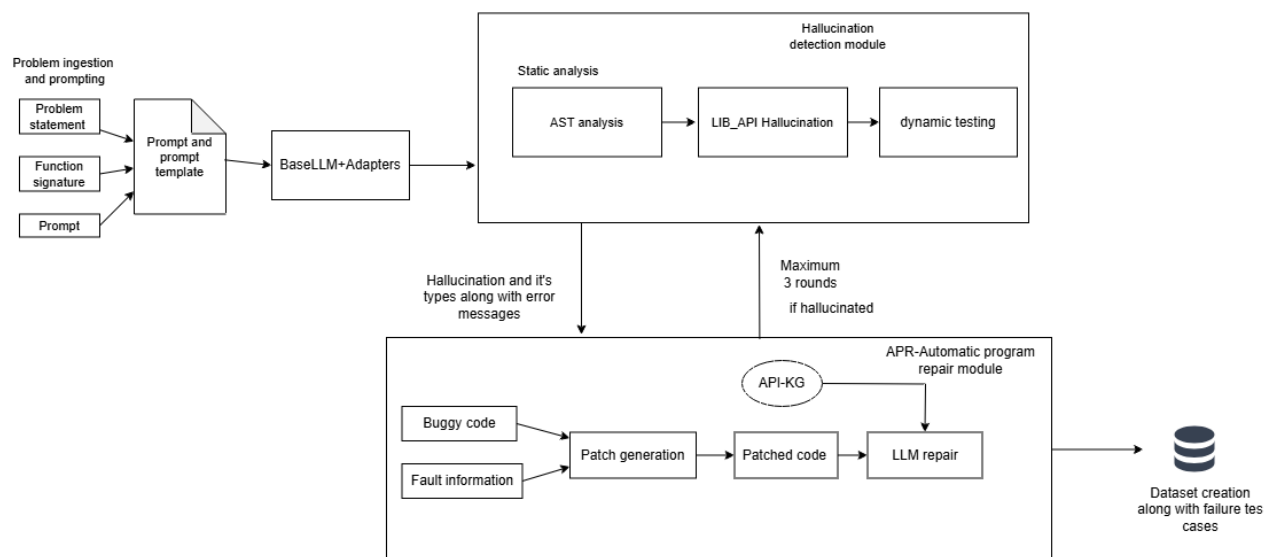


Fig1. Client side hallucination detection and mitigation framework

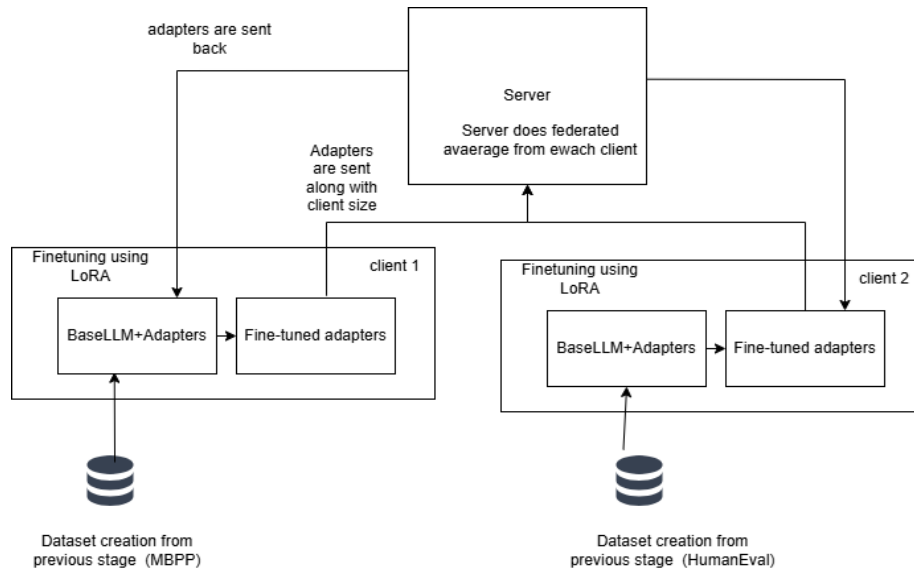


Fig 2. Federated learning + LoRA for fine-tuning

MODULES

There are two major phases in this project. Phase–1 (Fig.1) focuses on hallucination detection and mitigation in code generation. In this phase, the generated programs are analysed using both static and dynamic program analysis techniques to identify structural, semantic, and execution-level errors. When hallucinations are detected automatic program repair is done. During this phase prompt ,fault-information, buggy code are stored and this dataset is used for the next phase.

Phase–2 (Fig.2) performs PEFT in a federated learning environment. In this phase, each client locally trains lightweight LoRA from Phase–1. Instead of sharing raw code data, only the trained adapter parameters are transmitted to the central server. The server aggregates these adapters using federated averaging and redistributes the updated global adapter to the clients. The aggregated adapter is then combined with the base model to improve code generation quality in subsequent iterations.

1. Problem ingestion and code generation

Here, benchmark code generation datasets such as MBPP, HumanEval, and DS-1000 are loaded from Hugging Face datasets library. Code generation is done by single-shot prompting. The frozen Qwen2.5-Coder-3B pretrained model is used to generate the initial code solutions. These generated outputs may contain hallucinations and therefore serve as the baseline inputs for subsequent hallucination detection and analysis modules.

2. Hallucination detection

There are two stages in the hallucination detection framework: the static detection phase and the dynamic detection phase. The generated code from the code generation module is taken as input, and the module outputs detected hallucination types along with structured diagnostic information. In the static analysis stage, Abstract Syntax Tree (AST) is constructed to examine the structural representation of the program. This analysis is used to identify syntax errors, undefined variables, and invalid language constructs, allowing the detection of structural issues that may not be triggered during execution and thereby addressing limitations caused by incomplete test coverage across different branches of the program.

Then library analysis is performed using AST to check whether the functions and methods used in the generated code correctly align with the libraries available in the static execution environment.

In dynamic detection phase, code is executed within a controlled runtime environment using the test cases provided by the benchmark datasets. To ensure safe execution, the code is executed with timeout protection to prevent infinite loops. During execution, runtime exceptions and assertion failures are captured along with relevant information such as error type, error message, and line number obtained from traceback analysis. The framework also records the corresponding test cases, expected outputs, and actual outputs while validating against test cases. These execution results are structured and stored as diagnostic information, which is later used for hallucination classification and dataset construction for the fine-tuning stage.

3. Automatic program repair module

The automatic program repair module takes the input as buggy code along fault-information. Based on the fault information obtained from the previous modules, patch-based repair is applied by generating localized patches to the hallucinated code . The repair process is guided through fault-guided prompting, where fault information is provided to the LLM to assist in generating corrected code. The repair process is performed with a bounded number of attempts. Each repaired code is then verified using the hallucination detection module, and if verification fails after the bounded number of repair attempts, the code is marked as a hard hallucination and correct solution is fetched from the dataset.

4. Fine tuning using LoRA

Here we use Low-Rank Adaptation (LoRA)for fine-tuning to adapt the base LLM using dataset from previous stage while keeping the original model weights frozen. The input for this stage are prompt, corrected code, fault information and a frozen pre-trained LLM. The output of this stage is a set of client-specific LoRA adapters trained on the datasets. During training, weight updates are parameterized using two low-rank matrices A and B, where the rank r is set to 8 or 16. LoRA modules are injected into different projection matrices of each Transformer layer while keeping the original model parameters frozen and only the LoRA parameters trainable. After training, only the LoRA adapter weights are saved and transmitted to the central server. This approach enables each client to locally improve model performance using its own dataset while sharing only lightweight adapter, thereby preserving data privacy and contributing to the federated aggregation process.

5. Federated Aggregation and Distribution

To account for heterogeneous hallucination distributions across clients, the standard Federated Averaging algorithm can be extended with an frequency-weighted formula over traditional fed-average which only works over sample size. In this formulation, each client contributes to the global adapter based not only on the number of local training samples but also on the frequency of detected hallucination types. The aggregation weight therefore

combines dataset size and normalized error scores, enabling clients with more significant hallucination patterns to influence the global adapter more strongly. The server aggregates the client LoRA adapters using Federated Averaging(Frequency-Weighted Federated Averaging) to form a global adapter, which is then redistributed to all clients. Each client loads this global LoRA adapter together with the frozen base model, enabling improved code generation or further local fine-tuning without modifying the original model weights.

6. Evaluation and analysis

The evaluation and analysis phase is to measures the effectiveness of the proposed framework by comparing code generated by the base model and the federated LoRA-enhanced model across benchmark datasets. Functional correctness is evaluated using the Pass@1 metric, which measures whether the generated code passes all unit tests on the first attempt. Code similarity analysis is performed using CodeBLEU to assess the syntactic and structural similarity between the generated code and the reference implementations. Cross-client validation is conducted across federated clients to evaluate model generalization and robustness under heterogeneous data distributions. Finally, a before–after comparison is performed between the base model and the LoRA fine-tuned model to quantify improvements in functional correctness and reduction of hallucinations .

FORMULA

1. Basic LoRA

Let W_0 be the weight of the base model,

$$W_0 \in R^{d \times k}$$

Now we can breakdown this W_0 into two adapters B and A (LoRA matrices) with a rank r,

$$\Delta W = BA$$

Thus B and A are,

$$B \in R^{d \times r}, \quad A \in R^{r \times k}$$

Updated weights,

$$W = W_0 + BA$$

Scaled LoRA used at experiment,

$$W = W_0 + \frac{\alpha}{r} BA$$

r – rank of low-rank decomposition

α – scaling factor

2. Augmented Fed-average(Frequency-weighted)

Error frequency ,

$$F_e = \sum_{k=1}^K count_{k,e}$$

F_e = total occurrences of error type ‘ e ’ across all clients

$count_{k,e}$ = number of errors of type ‘ e ’ in client k

Weight Definition,

$$weight_e = \frac{F_e}{\sum_{e' \in errors} F_{e'}}$$

This is calculated for weight for each type of error based on their occurrence.

Error score,

For each client, the total count of each type of error along with its computed weight is accounted for

$$E_k = \sum_{e \in errors} count_{k,e} \cdot weight_e$$

Normalization,

$$\alpha_k = \frac{E_k}{\sum_{j=1}^K E_j}$$

Augmented FedAverage,

$$w^{t+1} = \sum_{k=1}^K \alpha_k w_k^t$$

w_k^t - local model weight on round t on client k

α_k - normalized weight assigned to client k

w^{t+1} - Global model weights after aggregation at round $t + 1$

EXPERIMENT

The experimental setup describes the computational environment, model configuration, and datasets used to implement and evaluate the proposed framework. Code generation and fine-tuning were done on Google Colab with T4 GPU, with 16 GB of memory.

Initial development, data preprocessing, and analysis were carried out in Visual Studio Code, after which the implementation was migrated to the Colab environment for GPU-based execution.

The base Model we use is QwenCoder-2.5-3B, a large language model containing approximately 3 billion parameters, 36 transformer layers. Training stages of the model are pre-training and post-training stages. Qwen has a strong capabilities in code generation tasks.

For evaluation, three benchmark datasets widely used in code generation research were utilized: HumanEval, MBPP, and DS-1000. HumanEval dataset consists of 164 programming problems with 5 associated features focusing on functional correctness. Most Basic Python Programming (MBPP) dataset contains 377 Python programming tasks with 6 features, focusing on basic programming and algorithmic problem-solving. The DS-1000 dataset comprises 1000 data science problems in Python, covering 6 libraries NumPy, Pandas, Matplotlib, SciPy, Sckit-learn, TensorFlow, PyTorch, statsmodel, seaborn and sympy.

1.Problem ingestion and code generation

During the code generation phase we used T4 GPU. Each dataset has a unique prompt template. The model was loaded using 4-bit quantization through the BitsAndBytes library, which reduces memory usage and allows large models to run efficiently. To maintain consistency and reproducibility across multiple runs, deterministic generation settings were applied by configuring `do_sample = False` and `temperature = 1.0`, ensuring that the model produces the same output for identical inputs during repeated executions.

2. Hallucination detection

Here static and dynamic analysis is done. Static analysis is followed by dynamic analysis. This specific idea is used from [1] where they have done SDHD. Each stage stores error information along with line number which will be used for patching the buggy code. Dynamic analysis is done using creating threads and running it in separate namespace. The thread handle timeout errors. The dynamic detection module which checks both working and semantic meaning of the code using testcases, if tests failed they are recorded as [input , actual output , expected output] triplets for further analysis and repair.

3. Automatic program repair module

After collecting fault information from the above module we generate patches (`<<<[START]` , `[END]>>>>`) for the buggy code. This makes the LLM to focus only on the buggy part and erroneous code segment. This helps LLM to perform good in APR [12]. APR is done 3 times ,if the problem still persists then we mark it as hard hallucination and we use canonical solution for dataset creation. The problem faced here is that QwenCoder-2.5-3B is a small LLM. There hasn't been a proper fix for the buggy code. This is still a model's inability to fix the code which we will mitigate during the fine-tuning phase.

Building a single pipeline for all the above, migrating from CSV driven code to a complete pipeline was a heavy job. Connecting to run-time, waiting for model-loading is a time-consuming and resource intensive task.

4. Fine tuning using LoRA

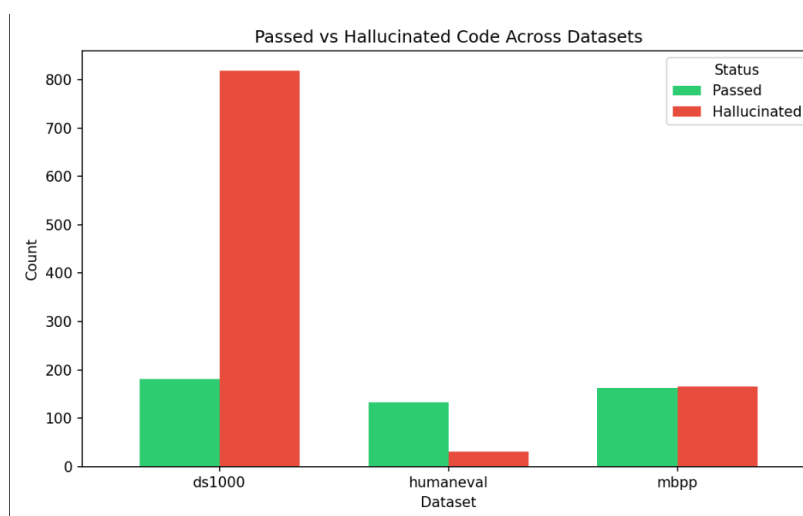
Since we have our current dataset. We start fine-tuning in this stage. We use T4 GPU. We apply LoRA a PEFT which is computationally smaller than full fine-tuning which updates every weight. The base model is frozen. Everything is quantised to 4-bits to make the result and development similar. Then LoRA adapters are made for all the 7 projections Q,K,V,O,up,down and gate. We fix the rank as 16 and alpha(scaling factor) as 32. Here we do SFT(supervised fine-tuning) with 3 epochs and an effective batch size of 8. In the end of fine-tuning we get .pt file which is tensors or adapter weights. We get a total of 504 adapters (36layers,2adapters each B,A for 7 projections) which is $36*2*2 = 504$.

This adapters are stored and used to cross-verify the improvement in performance. Once the base model is frozen and while using fine-tuned adapters the success rate went from 81% to 96% for humaneval dataset. This is a time-consuming job since it takes lot of time for model loading, adapter injection, code generation and analysis.

Future goal is to complete this process for MBPP and DS1000 dataset . Compare their performance of before fine tuning with base model and after fine-tuning. Also compare the results after Federated Averaging.

RESULTS AND ANALYSIS

Pass@1: The Pass@1 metric is used to measure functional correctness if code passes all unit tests on the first attempt.



ADAPTERS

```
=== Summary ===  
Number of LoRA parameters: 504  
Total elements: 29933568  
Parameter names (for FedAvg): ['base_model.model.model.layers.0.self_attn.q_proj.lora_A.default.weight', 'base_model.model.model
```

```
=== Attention projections ===  
Module      W shape      LoRA A shape  LoRA B shape  LoRA params  
-----  
k_proj      [256, 2048]   [16, 2048]    [256, 16]     36,864  
o_proj      [2048, 2048]  [16, 2048]    [2048, 16]    65,536  
q_proj      [2048, 2048]  [16, 2048]    [2048, 16]    65,536  
v_proj      [256, 2048]   [16, 2048]    [256, 16]     36,864  
  
=== MLP projections ===  
Module      W shape      LoRA A shape  LoRA B shape  LoRA params  
-----  
down_proj   [2048, 11008] [16, 11008]   [2048, 16]    208,896  
gate_proj   [11008, 2048] [16, 2048]    [11008, 16]   208,896  
up_proj     [11008, 2048] [16, 2048]    [11008, 16]   208,896
```

Pass@1 rate comparison for HumanEval dataset before and after LoRA fine-tuning.

```
... === Pass rate comparison ===  
Before SFT (from CSV): 81.10% (133/164)  
After SFT (adapters): 96.15% (125/130)  
Difference: +15.06%  
Conclusion: Adapter improves HumanEval pass@1.
```

REFERENCES

- [1] B. Yang, J. Dang, H. Liu and Z. Jin, "Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection" in *IEEE Transactions on Software Engineering*, vol. , no. 01, pp. 1-17, PrePrints 5555, doi: 10.1109/TSE.2025.3640641.
- [2] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty and S. K. Lahiri, "LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation," in *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2254-2268, Sept. 2024, doi: 10.1109/TSE.2024.3428972.
- [3] J. Chen, Z. Cai, W. Chen, W. Wang, Z. Zheng and P. S. Yu, "A Federated Adaptive Large Language Model Fine-Tuning Framework for Software Development," in *IEEE Transactions on Services Computing*, doi: 10.1109/TSC.2025.3623626
- [4] M. Nashaat and J. Miller, "Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review," in *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2240-2253, Sept. 2024, doi: 10.1109/TSE.2024.3428324
- [5] R. Khojah, F. G. de Oliveira Neto, M. Mohamad and P. Leitner, "The Impact of Prompt Programming on Function-Level Code Generation," in *IEEE Transactions on Software Engineering*, vol. 51, no. 8, pp. 2381-2395, Aug. 2025, doi: 10.1109/TSE.2025.3587794
- [6] Q. Wang, C. Li, Y. Liu, Q. Zhu, J. Song and T. Shen, "An Adaptive Framework Embedded With LLM for Knowledge Graph Construction," in *IEEE Transactions on Multimedia*, vol. 27, pp. 2912-2923, 2025, doi: 10.1109/TMM.2025.3557717
- [7] X. Yu, L. Liu, X. Hu, J. W. Keung, J. Liu and X. Xia, "Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?," in *IEEE Transactions on Software Engineering*, vol. 50, no. 12, pp. 3435-3453, Dec. 2024, doi: 10.1109/TSE.2024.3492204
- [8] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, "FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM," in *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 51854-51865, 15 Dec.15, 2025, doi: 10.1109/IJOT.2025.3582427

- [9] Z. Liu, Y. Tang, X. Luo, Y. Zhou and L. F. Zhang, "No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT," in *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548-1584, June 2024, doi: 10.1109/TSE.2024.3392499
- [10] S. Ouyang, J. M. Zhang, Z. Sun and A. M. Penuela, "Knowledge-Enhanced Program Repair for Data Science Code," in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, 2025, pp. 898-910, doi: 10.1109/ICSE55347.2025.00246.
- [11] J. Liu, Y. Zhang, D. Wang, Y. Li and W. Dong, "THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge," 2025 *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Montreal, QC, Canada, 2025, pp. 229-240, doi: 10.1109/SANER64311.2025.00029.
- [12] H. Zhang, Q. Yan, W. Min, C. Zhang, L. Kuang and Y. Xia, "EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA," 2025 *IEEE International Conference on Web Services (ICWS)*, Helsinki, Finland, 2025, pp. 946-956, doi: 10.1109/ICWS67624.2025.00123.
- [13] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, "FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM," in *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 51854-51865, 15 Dec.15, 2025, doi: 10.1109/JIOT.2025.3582427